

HDOC DAY-28 ASSIGNMENT

Variable-Length 2D Arrays and Row-Major Operations in C

QUESTION 1

Read a variable-length 2D array column by column and print it in row-major order using a single loop.

1. Problem Understanding

The program reads the number of rows and columns and repeatedly asks for them until both are positive. It then declares a variable-length 2D integer array. The array elements are entered column by column using nested loops with the outer loop over columns and the inner loop over rows. Finally, the array is displayed in row-major order using one loop and a linear index.

2. Standard Test Cases

Test Case	Rows	Columns	Elements entered column-wise	Expected row-major output	Status
TC1	2	3	1, 4, 2, 5, 3, 6	1 2 3 / 4 5 6	Pass
TC2	3	2	1, 3, 5, 2, 4, 6	1 2 / 3 4 / 5 6	Pass
TC3	1	4	7, 8, 9, 10	7 8 9 10	Pass

3. Algorithm

1. Start.
2. Read rows and columns.
3. If rows or columns are not positive, repeat the input until both values are positive.
4. Declare a variable-length 2D integer array using the given rows and columns.
5. Use an outer loop for columns and an inner loop for rows to read each array element column by column.
6. Set a linear index $k = 0$.
7. Repeat while $k < \text{rows} * \text{columns}$.
8. Find the row index as $k / \text{columns}$ and the column index as $k \% \text{columns}$.
9. Print `array[k / columns][k % columns]`.
10. After every complete row, print a newline.
11. Increase k by 1 and continue until all elements are displayed.
12. Stop.

4. C Program

```
#include <stdio.h>
int main()
{
    int rows, cols, i, j, k;
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        printf("Enter number of columns: ");
        scanf("%d", &cols);
```

```

    if (rows <= 0 || cols <= 0)
        printf("Rows and columns must be positive. Enter again.\n");
    }
    while (rows <= 0 || cols <= 0);
    int a[rows][cols];
    printf("Enter array elements column by column:\n");
    for (j = 0; j < cols; j++)
    {
        for (i = 0; i < rows; i++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }
    printf("Array in row-major order:\n");
    for (k = 0; k < rows * cols; k++)
    {
        printf("%d ", a[k / cols][k % cols]);
        if ((k + 1) % cols == 0)
            printf("\n");
    }
    return 0;
}

```

5. Compilation and Execution Evidence



```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 2 3
Enter number of columns: Enter array elements column by column:
a[0][0] = 1 2 4 5 3 6
a[1][0] = a[0][1] = a[1][1] = a[0][2] = a[1][2] = Array in row-major order:
1 4 3
2 5 6

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 3 2
Enter number of columns: Enter array elements column by column:
a[0][0] = 1 3 5 2 4 6
a[1][0] = a[2][0] = a[0][1] = a[1][1] = a[2][1] = Array in row-major order:
1 2
3 4
5 6

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 1 4
Enter number of columns: Enter array elements column by column:
a[0][0] = 7 8 9 10
a[0][1] = a[0][2] = a[0][3] = Array in row-major order:
7 8 9 10

(kali@kali)~/Desktop
$

```

6. Verification of All Test Cases

Test Case	Input dimensions	Input elements	Expected result	Actual result	Status
TC1	2 x 3	1,4,2,5,3,6	1 2 3 / 4 5 6	1 2 3 / 4 5 6	Pass
TC2	3 x 2	1,3,5,2,4,6	1 2 / 3 4 / 5 6	1 2 / 3 4 / 5 6	Pass
TC3	1 x 4	7,8,9,10	7 8 9 10	7 8 9 10	Pass

QUESTION 2

Read a variable-length 2D array, copy it into a 1D array in row-major order, and display both arrays.

1. Problem Understanding

The program reads positive row and column values, creates a variable-length 2D integer array, and accepts all its elements. It then copies the 2D array into a 1D array in row-major order, so the elements of the first row are copied first, followed by the second row, and so on. Finally, both the 2D array and the resulting 1D array are displayed.

2. Standard Test Cases

Test Case	Rows	Columns	2D array input (row-wise)	Expected 1D array	Status
TC1	2	3	1 2 3 / 4 5 6	1 2 3 4 5 6	Pass
TC2	3	2	10 20 / 30 40 / 50 60	10 20 30 40 50 60	Pass
TC3	2	2	7 8 / 9 10	7 8 9 10	Pass

3. Algorithm

1. Start.
2. Read rows and columns.
3. If rows or columns are not positive, repeat the input until both values are positive.
4. Declare a variable-length 2D integer array and a 1D integer array of size rows * columns.
5. Read all elements of the 2D array row by row using nested loops.
6. Set k = 0.
7. Traverse the 2D array row by row.
8. Copy each element into the 1D array as `oneD[k] = a[i][j]`, then increase k by 1.
9. Display the 2D array using nested loops.
10. Display all elements of the 1D array using a single loop.
11. Stop.

4. C Program

```
#include <stdio.h>
int main()
{
    int rows, cols, i, j, k = 0;
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        printf("Enter number of columns: ");
        scanf("%d", &cols);
        if (rows <= 0 || cols <= 0)
            printf("Rows and columns must be positive. Enter again.\n");
    }
    while (rows <= 0 || cols <= 0);
```

```

int a[rows][cols];
int oneD[rows * cols];
printf("Enter elements of the 2D array:\n");
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        oneD[k] = a[i][j];
        k++;
    }
}
printf("2D array:\n");
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
        printf("%d ", a[i][j]);
    printf("\n");
}
printf("1D array in row-major order:\n");
for (k = 0; k < rows * cols; k++)
    printf("%d ", oneD[k]);
printf("\n");
return 0;
}

```

5. Compilation and Execution Evidence

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 2
Enter number of columns: 3
Enter elements of the 2D array:
a[0][0] = 1
a[0][1] = 2
a[0][2] = 3
a[1][0] = 4
a[1][1] = 5
a[1][2] = 6
2D array:
1 2 3
4 5 6
1D array in row-major order:
1 2 3 4 5 6

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 3
Enter number of columns: 2
Enter elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[1][0] = 30
a[1][1] = 40
a[2][0] = 50
a[2][1] = 60
2D array:
10 20
30 40
50 60
1D array in row-major order:
10 20 30 40 50 60

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 2
Enter number of columns: 2
Enter elements of the 2D array:
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

1 2 3
4 5 6
1D array in row-major order:
1 2 3 4 5 6

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 3
Enter number of columns: 2
Enter elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[1][0] = 30
a[1][1] = 40
a[2][0] = 50
a[2][1] = 60
2D array:
10 20
30 40
50 60
1D array in row-major order:
10 20 30 40 50 60

(kali@kali)~/Desktop
$ ./question
Enter number of rows: 2
Enter number of columns: 2
Enter elements of the 2D array:
a[0][0] = 7
a[0][1] = 8
a[1][0] = 9
a[1][1] = 10
2D array:
7 8
9 10
1D array in row-major order:
7 8 9 10

(kali@kali)~/Desktop
$
```

6. Verification of All Test Cases

Test Case	Input dimensions	2D input	Expected 1D	Actual 1D	Status
TC1	2 x 3	1 2 3 / 4 5 6	1 2 3 4 5 6	1 2 3 4 5 6	Pass
TC2	3 x 2	10 20 / 30 40 / 50 60	10 20 30 40 50 60	10 20 30 40 50 60	Pass
TC3	2 x 2	7 8 / 9 10	7 8 9 10	7 8 9 10	Pass

